EcoRuteando: diseño e implementación de una plataforma de movilidad urbana sostenible con información geográfica para Neiva, Colombia

EcoRuteando: Design and Implementation of a Sustainable Urban Mobility Platform Using Geographic Information for Neiva, Colombia

Autor: Daniel Salazar Vargas

RESUMEN

El incremento del parque automotor en ciudades intermedias de América Latina genera impactos significativos en la congestión vial, la contaminación atmosférica y la salud pública. Este artículo presenta el diseño e implementación de EcoRuteando, una plataforma de software orientada a la movilidad urbana sostenible en la ciudad de Neiva, Huila, Colombia. La solución integra una arquitectura backend modular construida sobre .NET 9 con el patrón CQRS-lite, una base de datos relacional y geoespacial en PostgreSQL 16 con PostGIS 3.5 desplegada en la plataforma Render, y una interfaz web SPA desarrollada en React desplegada en la plataforma Vercel. El sistema procesa peticiones de ruteo exclusivamente para modos de transporte terrestre admisibles: vehículo particular y desplazamiento a pie (caminata). Con base en la distancia del trayecto, estima la mitigación teórica de dióxido de carbono (CO_2) mediante una función almacenada determinista basada en el escenario de referencia del automóvil frente al desplazamiento peatonal. La infraestructura soporta tanto orquestación local mediante Docker Compose como un esquema de despliegue en la nube (Vercel/Render) y gestión en su capa de datos por migraciones automáticas con Liquibase 5.0.2. Se analizan el estado funcional de sus componentes, la decisión de arquitectura (ADR-8) que restringió el motor de ruteo a vehículos particulares y caminata, el mecanismo de auditoría de datos y las limitaciones derivadas de la ausencia de pruebas de carga masiva y medición directa de emisiones.

Palabras clave: Movilidad urbana sostenible, Sistemas de Información Geográfica, PostGIS, .NET 9, Vercel, Render, Factores de emisión de CO_2, Neiva.

ABSTRACT

The growth of the vehicle fleet in intermediate Latin American cities generates significant impacts on traffic congestion, air pollution, and public health. This paper presents the design and implementation of EcoRuteando, a software platform aimed at sustainable urban mobility in the city of Neiva, Huila, Colombia. The solution integrates a modular backend architecture built on .NET 9 using the CQRS-lite pattern, a relational and geospatial database in PostgreSQL 16 with PostGIS 3.5 deployed on the Render cloud platform, and a React SPA web interface deployed on Vercel. The system processes routing requests exclusively for supported land transport modes: private vehicles and walking (pedestrians). Based on the route distance, it estimates theoretical carbon dioxide (CO_2) mitigation using a deterministic stored function based on the private car baseline scenario versus pedestrian movement. The infrastructure supports both local containerized orchestration via Docker Compose and cloud deployment (Vercel/Render) with automated Liquibase 5.0.2 migrations at its data layer. The functional status of its components, the architectural decision (ADR-8) restricting the routing engine to private vehicles and walking, the data audit mechanism, and the limitations arising from the absence of massive load testing and direct emission measurements are analyzed.

Keywords: Sustainable urban mobility, Geographic Information Systems, PostGIS, .NET 9, Vercel, Render, CO_2 emission factors, Neiva.

I. INTRODUCCIÓN

En las últimas décadas, las ciudades intermedias de los países en desarrollo han experimentado un crecimiento acelerado en su tasa de motorización privada. Este fenómeno, impulsado por el incremento de automóviles particulares y motocicletas frente a deficiencias en la infraestructura vial, desencadena problemas de congestión, accidentalidad y emisión de gases de efecto invernadero (GEI), principalmente dióxido de carbono (CO_2). En Colombia, el municipio de Neiva (capital del departamento del Huila) presenta una temperatura promedio elevada y una dependencia marcada del transporte motorizado, lo que incrementa la huella ambiental por desplazamientos cotidianos.

Frente a este escenario, la movilidad urbana sostenible promueve incentivar los desplazamientos activos (como la caminata) y optimizar el uso de vehículos para reducir las externalidades ambientales negativas. Para respaldar esta transición, se requieren herramientas tecnológicas transparentes que informen no solo sobre la distancia o el tiempo de trayecto, sino también sobre el impacto ambiental estimado al optar por desplazarse a pie en lugar de utilizar un vehículo de combustión.

Aunque existen plataformas globales de navegación cartográfica, estas herramientas están enfocadas principalmente en optimizar el tiempo de viaje para vehículos particulares y no cuantifican la mitigación teórica de CO_2 en el contexto local ni ofrecen canales directos de reporte comunitario sobre obstáculos en las vías.

Este artículo documenta la arquitectura, diseño e implementación de EcoRuteando, un sistema de información geográfica y plataforma web orientada a la movilidad en Neiva. El objetivo general es desarrollar una infraestructura tecnológica desacoplada, extensible y adaptable tanto a contenedores locales como a entornos de nube (Vercel y Render) que permita calcular trayectos urbanos para vehículos particulares y peatones, gestionar reportes de incidencias viales y estimar la mitigación teórica de CO_2 generada cuando un trayecto se realiza a pie en comparación con el escenario de referencia del automóvil. Se analizan la estructura de sus componentes (.NET 9, React, PostgreSQL/PostGIS, Vercel, Render, Docker), la trazabilidad de sus requisitos y las limitaciones asociadas a su estado actual de desarrollo y validación.

II. FUNDAMENTACIÓN TEÓRICA Y TRABAJOS RELACIONADOS

A. Sistemas de Información Geográfica (SIG) y Ruteo Urbano

Los Sistemas de Información Geográfica (SIG) constituyen el pilar tecnológico para el almacenamiento, consulta y análisis de datos espaciales. La integración de PostgreSQL con la extensión espacial PostGIS permite almacenar objetos geométricos bajo estándares del Open Geospatial Consortium (OGC), especificando sistemas de referencia espacial como el World Geodetic System 1984 (WGS 84, identificado con SRID 4326). El ruteo urbano en SIG representa la malla vial mediante aristas y nodos con pesos asociados a distancia o tiempo.

B. Estimación de Emisiones de CO_2 por Fuentes Móviles

La cuantificación de emisiones de gases de efecto invernadero en el transporte terrestre se realiza mediante dos enfoques principales:

Medición directa en tubo de escape (sensores PEMS): Ofrece alta precisión pero exige equipamiento físico inviable para monitoreo masivo en tiempo real.

Estimación basada en factores de emisión por actividad: Aplica ecuaciones deterministas que multiplican la distancia recorrida (kilómetros) por un factor de emisión estandarizado ($\text{kg CO}_2/\text{km}$) asociado a la categoría vehicular.

En EcoRuteando, el cálculo de CO_2 mitigado o "ahorrado" se define formalmente como la emisión teórica que habría generado un desplazamiento en automóvil particular frente a la emisión nula ($0.000 \text{ kg CO}_2/\text{km}$) que representa la caminata.

C. Trabajos Relacionados

Diversas investigaciones han explorado plataformas de ruteo verde (green routing) y sistemas de gestión de movilidad en entornos urbanos. Sin embargo, la mayoría de los sistemas comerciales no adaptan sus coeficientes de emisión a las realidades del parque automotor de ciudades intermedias ni integran esquemas transparentes de datos abiertos y auditoría. EcoRuteando aborda esta brecha mediante un diseño modular con almacenamiento auditado en PostgreSQL/PostGIS.

III. METODOLOGÍA Y ARQUITECTURA DE SOFTWARE

A. Enfoque Metodológico

Se adoptó un enfoque de ingeniería de software empírico y estructurado articulado en cuatro fases:

Análisis y Especificación de Requisitos: Levantamiento de Requisitos Funcionales (RF) y No Funcionales (RNF) bajo la norma IEEE 29148.

Diseño Arquitectónico y Modelado de Datos: Estructuración en capas limpias (Clean Architecture / DDD) y definición del esquema relacional en PostgreSQL 16.

Implementación e Integración: Desarrollo del backend en C# (.NET 9), frontend en React (Vite) y orquestación de despliegue en la nube mediante Vercel (Frontend SPA) y Render (Web Service API y Managed PostgreSQL con PostGIS).

Auditoría y Verificación: Contrastación directa de funcionalidades contra el código fuente y la documentación técnica.

B. Arquitectura y Tecnologías Verificadas

La arquitectura de EcoRuteando responde a una topología desacoplada de tres capas orquestada en la nube mediante Vercel para la capa de cliente y Render para la capa de servicios y persistencia.

Plaintext

Figura 1. Diagrama de arquitectura física y despliegue en la nube (Vercel / Render) de EcoRuteando.

Componentes del Backend:

Framework: .NET 9 (ASP.NET Core Web API) en C#, alojado como Web Service en Render.

Patrón Organizacional: Domain-Driven Design (DDD) modular con patrón CQRS-lite mediante la librería MediatR.

Persistencia: Entity Framework Core 9 con el proveedor Npgsql.EntityFrameworkCore.PostgreSQL.

Seguridad: Autenticación con JSON Web Tokens (JWT) firmados, algoritmos de hashing BCrypt.Net para contraseñas y middleware de manejo global de excepciones.

Componentes del Frontend:

Stack: React con Vite y Tailwind CSS ejecutándose sobre Node.js 22, desplegado y servido globalmente a través de la infraestructura CDN de Vercel.

Integración API: Configuración de variables de entorno de producción para comunicación HTTPS directa con el endpoint del backend en Render, resolviendo problemas de CORS mediante

encabezados HTTP permitidos.

Internacionalización: Módulo react-i18next con soporte para español (es), inglés (en), portugués (pt) y francés (fr).

Integraciones con Servicios Externos Verificados:

Google Maps Platform / OpenStreetMap: Integrados para renderizado de capas cartográficas y geocodificación de coordenadas.

OpenWeather API: Servicio de consulta de estado meteorológico consumido por la API.

Gmail SMTP: Configurado en el puerto 587 (TLS/STARTTLS) en la capa de infraestructura backend para envío de correos transaccionales de verificación y recuperación.

C. Requisitos y Modelado de Datos

La base de datos ecoRuteando_db se estructura en tablas normalizadas con soporte PostGIS (desplegada en Render PostgreSQL) y migraciones automáticas mediante Liquibase 5.0.2 (driver JDBC postgresql-42.7.8).

Tablas Principales:

Seguridad y Usuarios: usuarios, seg_roles, seg_permisos, seg_usuario_rol, seg_sesiones, seg_auditoria, oauth_cuentas.

Movilidad y Rutas: rutas, historial_direcciones, uso_rutas, rutas_favoritas, calificaciones, puntos_interes.

Comunidad: reporte_obstaculos, reporte_votos, alertas.

Tipos Espaciales e Índices:

Las coordenadas de origen, destino y reportes se almacenan utilizando la columna tipo GEOMETRY(Point, 4326). Las trayectorias de ruteo para vehículos y peatones se representan con GEOMETRY(LineString, 4326). Se crearon índices GIST en todas las columnas geométricas (idx_rutas_geom_inicio, idx_reportes_ubicacion) e índices GIN con pg_trgm para búsquedas de texto.

D. Modelo de Estimación de Emisiones (CO_2)

El cálculo de la mitigación de CO_2 se implementa en la base de datos mediante la función PL/pgSQL fn_calcular_co2(p_distancia_km, p_tipo_transporte).

Formulación Matemática del Modelo:

$$\text{CO}_{2,\mathrm{evitado}} = d \times \left(FE_{\mathrm{automóvil}} - FE_{\mathrm{caminata}} \right)$$

Donde:

$\text{CO}_{2,\mathrm{evitado}}$: Masa teórica de dióxido de carbono mitigada o evitada, expresada en kilogramos (kg CO_2).

d: Distancia total calculada para el trayecto, expresada en kilómetros (km), derivada de funciones de la extensión espacial PostGIS.

$FE_{\mathrm{automóvil}}$: Factor de emisión de referencia asignado al desplazamiento en automóvil particular de combustión interna, expresado en $\text{kg CO}_2/\text{km}$.

FE_{caminata}: Factor de emisión directo asociado a la caminata peatonal, definido como $0.000 \text{ kg CO}_2/\text{km}$ (ausencia de emisiones por escape).

Factores de Emisión Configurados en el Sistema:

En los scripts de inicialización y en la tabla configuracion_sistema (poblados mediante dml.sql), los coeficientes base por defecto asignados son:

Automóvil Particular ($FE_{\mathrm{automóvil}}$): $0.192 \text{ kg CO}_2/\text{km}$ (Parámetro de comparación por defecto).

Caminata Peatonal (FE_{caminata}): $0.000 \text{ kg CO}_2/\text{km}$.

Supuesto operativo del cálculo: Cuando un usuario solicita una ruta en modo peatonal, la función fn_calcular_co2 evalúa $d \times (0.192 - 0.000)$, asignando como ahorro la totalidad de la emisión que habría generado el automóvil. Si el ruteo se realiza para vehículo particular, la emisión mitigada calculada es 0.000 kg CO_2.

IV. RESULTADOS Y DISCUSIÓN

A. Implementación Web y Despliegue de la API

El backend en .NET 9 desplegado en Render expone una API REST con contratos documentados mediante OpenAPI / Swagger UI. La SPA de React alojada en Vercel consume estos servicios de forma segura. La siguiente tabla unificada resume los endpoints principales cuya existencia y estructura de controladores fueron verificadas en el código fuente:

B. Estado de la Aplicación Móvil

La aplicación móvil fue desarrollada utilizando el marco de trabajo React Native (código fuente localizado en MovilEcoRuteando.zip), integrando almacenamiento seguro de secretos con SecureStore y visores de mapas. Se clasifica formalmente como un prototipo funcional en fase de integración. A diferencia de la aplicación web SPA alojada en Vercel, la app móvil requiere compilación independiente para distribución en tiendas de aplicaciones.

C. Gestión de Información Geográfica y Reportes Ciudadanos

Se confirmó el funcionamiento del esquema para incidencias comunitarias. Al registrar un reporte en reporte_obstaculos, PostGIS almacena la localización geográfica (POINT) con el estado PENDIENTE. Los usuarios con rol MODERADOR o ADMINISTRADOR pueden aprobar o rechazar el evento mediante el panel de control.

Mecanismo de Auditoría de Datos:

Se verificó la definición del trigger PL/pgSQL trg_audit_usuarios sobre la entidad seg_auditoria. Frente a eventos de modificación (UPDATE) o eliminación (DELETE), el sistema almacena las estructuras de datos anterior y posterior en formato binario JSONB junto con el sello de tiempo.

SQL

```
CREATE OR REPLACE FUNCTION fn_audit_usuarios()
RETURNS TRIGGER AS $$
BEGIN
INSERT INTO seg_auditoria(id_auditoria, tabla_afectada, operacion, datos_anteriores, datos_nuevos, fecha_registro)
VALUES (gen_random_uuid(), 'usuarios', TG_OP, to_jsonb(OLD), to_jsonb(NEW), NOW());
RETURN NEW;
END;
```

$$ LANGUAGE plpgsql;

D. Estimación de Emisiones de CO_2 y Evaluación Numérica

Para verificar la consistencia determinista del algoritmo, se aplicó la función fn_calcular_co2 sobre un trayecto peatonal evaluado de 3.5 km dentro del perímetro urbano de Neiva. La evaluación produce el siguiente resultado teórico:

$$\text{CO}_{2,\mathrm{evitado}} = 3.5 \text{ km} \times \left(0.192 \text{ kg CO}_2/\text{km} - 0.000 \text{ kg CO}_2/\text{km} \right) = 0.672 \text{ kg CO}_2$$

Este cálculo demuestra la coherencia matemática interna de la función almacenada. Se enfatiza que este resultado representa una aproximación teórica educativa y no una medición física directa de contaminantes reducidos en la atmósfera de la ciudad.

E. Limitaciones y Decisiones Arquitectónicas (ADR-8)

Durante la auditoría del proyecto se identificaron decisiones de alcance e infraestructura esenciales:

Restricción del Alcance de Modos de Transporte (ADR-8): En las fases documentales preliminares se contempló el ruteo de bicicletas y autobuses urbanos. No obstante, la Decisión de Arquitectura ADR-8 (Septiembre 2026) defined que la versión implementada de la API REST operaría de manera exclusiva sobre vehículos particulares y peatones (caminata).

Ausencia de Pruebas de Carga Concurrente y Evaluaciones de Usabilidad: No se identificaron bitácoras de pruebas masivas de esfuerzo (ej. Apache JMeter) ni estudios estadísticos de uso con ciudadanos de Neiva. Por lo tanto, no se asumen latencias bajo alta demanda ni reducciones cuantitativas demostradas en la congestión vial de la ciudad.

V. CONCLUSIONES Y TRABAJO FUTURO

A. Conclusiones

Se diseñó e implementó la plataforma EcoRuteando, integrando una API modular en .NET 9, base de datos relacional y geoespacial en PostgreSQL 16 / PostGIS 3.5 en Render y un cliente web SPA en React desplegado en Vercel.

El motor de ruteo procesa trayectos para vehículos particulares y peatones, integrando en la base de datos la función fn_calcular_co2 para estimar el CO_2 teóricamente mitigado al realizar desplazamientos a pie frente al uso del automóvil.

La arquitectura en PostgreSQL con triggers PL/pgSQL y columnas JSONB provee un mecanismo de auditoría estructurado para la trazabilidad de datos en entidades sensibles.

La clasificación transparente del software permitió identificar la aplicación Web SPA como plenamente funcional y el cliente en React Native como un prototipo en desarrollo.

B. Trabajo Futuro

Inclusión de Modos Adicionales de Transporte: Evaluar en etapas posteriores la incorporación de redes de transporte público masivo y carriles para bicicletas.

Pruebas de Carga y Rendimiento: Ejecutar pruebas de estrés automatizadas para determinar la latencia y tasa de transferencia de la API en Render ante peticiones concurrentes.

Calibración de Coeficientes de Emisión: Ajustar el parámetro $0.192 \text{ kg CO}_2/\text{km}$ mediante estudios de caracterización del parque automotor local en Neiva.

Empaquetado de la App Móvil: Configurar procesos de integración continua para la compilación de binarios Android (APK) del cliente React Native.

REFERENCIAS BIBLIOGRÁFICAS

[1] ISO/IEC/IEEE, "Systems and software engineering — Life cycle processes — Requirements engineering," ISO/IEC/IEEE 29148:2018, pp. 1-104, 2018.

[2] OGC, "OpenGIS Implementation Standard for Geographic information - Simple feature access - Part 1: Common architecture," Open Geospatial Consortium, Tech. Rep. OGC 06-103r4, 2011.

[3] E. E. Environmental Agency, "EMEP/EEA air pollutant emission inventory guidebook 2019: Technical guidance to prepare national emission inventories," European Environment Agency, Luxembourg, EUR-OP, 2019.

[4] PostgreSQL Global Development Group, "PostgreSQL 16.0 Documentation," PostgreSQL Docs, 2023. [Online]. Available: https://www.postgresql.org/docs/16/index.html

[5] PostGIS Project, "PostGIS 3.5.0 Manual," Refraction Research, 2024. [Online]. Available: https://postgis.net/documentation/

[6] Microsoft, ".NET 9 Architectural Guide and ASP.NET Core Web API Best Practices," Microsoft Learn, 2024. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/

Endpoint	Método HTTP	Descripción	Estado de Verificación
/api/v1/auth/login	POST	Autentica usuarios mediante credenciales, emite tokens JWT firmados y gestiona el bloqueo de cuentas tras 5 intentos fallidos consecutivos.	🟢 Implementado en código
/api/v1/rutas/calcular	POST	Procesa coordenadas de origen/destino para vehículos particulares y peatones, calculando la distancia espacial y el CO_2 evitado mediante fn_calcular_co2.	🟢 Implementado en código
/api/v1/reportes	POST / GET	Permite el registro comunitario y la consulta de obstáculos en las vías asociando puntos geomorfométricos en PostGIS (GEOMETRY).	🟢 Implementado en código
/api/v1/admin/dashboard	GET	Recupera indicadores consolidados de uso y mitigación teórica consultando la vista de base de datos vista_estadisticas_globales.	🟢 Implementado en código